Formato para Presentar Proyectos Modulares (INNI) "Fraud-AI"

Diego Iván Hernández Muñóz, Luis Angel De La Cruz Ascencio, Dr. Jorge de Jesús Gálvez Rodríguez

CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS, (CUCEI, UDG)

diego.hernandez5523@alumnos.udg.mx
luis.delacruz9064@alumnos.udg.mx
jorge.galvez@academicos.udg.mx

FIRMA DE VISTO BUENO DEL ASESOR

***Abstract*— El fraude en transacciones bancarias es un problema crítico y persistente en la actualidad. El proyecto modular desarrollado, titulado Fraud-AI, es un sistema SaaS orientado a comercios pequeños y medianos que permite el monitoreo y la detección de posibles transacciones fraudulentas. Para lograrlo, se implementaron modelos de IA supervisados, como Random Forest y Regresión Logística, junto con un enfoque no supervisado mediante el algoritmo K-Means. Con estos algoritmos se llevó a cabo un aprendizaje utilizando un dataset de trescientos mil transacciones simuladas, el cual permitió que el sistema pudiera determinar sus propias decisiones basadas en los datos aprendidos.**

Palabras claves - Detección de fraude, Machine Learning, comercio electrónico, SaaS, transacciones financieras, modelos híbridos, Regresión Logística, Random Forest, K-Means, monitoreo en tiempo real, metodología SCRUM, chargeback, explicabilidad, SHAP, HTTP Rest, JWT, JSON.

Repositorio de código: https://github.com/Ivanpillow/Fraud-AI
Versión actual de código: 1.0
Licencia legal código: Propietario

I. INTRODUCCIÓN

Este documento se encarga de detallar todo el proceso llevado a cabo para desarrollar el proyecto modular titulado "Fraud-AI". La idea surgió a partir de la problemática que se encuentra presente durante el día a día en miles de negocios electrónicos, el fraude. Las transacciones fraudulentas han ido tomando un creciente impacto conforme va aumentando la digitalización, es decir, a mayor oferta, mayor consumo.

Mientras los negocios, año con año, deciden adoptar un enfoque tecnológico para la implementación de compras en línea de sus productos o servicios, también crecen las posibles evasiones a la ley mediante compras fraudulentas, amenazando el crecimiento económico del negocio.

Aunque en una primera instancia se podría pensar que a los negocios no les importa si las compras que se realizan en sus plataformas son transacciones fraudulentas o no, ya que de igual forma se encuentran generando ingresos, la realidad es que el problema va más allá. Los dueños originales de las tarjetas de crédito o débito, tarde o temprano, se darían cuenta de que se realizó una o varias compras no autorizadas utilizando sus datos financieros, por lo que comúnmente optan por comunicarse directamente con su banco para anular dichas transacciones.

Una vez llevada a cabo la serie de pasos correspondientes, se procede a realizar un "chargeback", el cual consiste en la anulación de un pago a través del banco. En un nivel básico, los fondos se retiran de la cuenta del comercio y se devuelven al cliente, afectando directamente la economía del negocio al perder el ingreso de la venta, el producto enviado, incrementar su tasa de fraude y aumentar el riesgo de que organizaciones como Visa o MasterCard lo clasifiquen como un comercio de alto riesgo.

Es por ello por lo que se decidió desarrollar Fraud-AI, un sistema SaaS (Software as a Service) pensado para su implementación en negocios electrónicos, con el fin de que dichos comercios tengan la posibilidad de validar por sí mismos aquellas compras que el sistema detecte mediante un modelo híbrido, el cual combina reglas heurísticas y técnicas de Machine Learning.

Los comercios, al implementar el servicio de Fraud-AI, tendrían la posibilidad de observar y analizar el histórico de transacciones realizadas en su negocio, así como una serie de gráficos que recopilan información relevante relacionada con ganancias, comportamiento de clientes y métricas clave de operación. Además, el sistema incorporaría un módulo de monitoreo que muestre de forma clara las compras que Fraud-AI haya bloqueado por posible fraude, así como aquellas que hayan sido catalogadas como "en revisión", permitiendo al comercio tomar una decisión final de manera informada.

Los objetivos de la solución son los siguientes:

1. Proporcionar a los comercios un mayor control sobre las transacciones fraudulentas realizadas en su negocio, permitiéndoles identificar y gestionar posibles riesgos antes de que impacten económicamente.
2. Mostrar diversos gráficos basados en datos reales, que faciliten la visualización del comportamiento de las transacciones, métricas de fraude, ingresos y patrones de consumo.
3. Detectar de manera temprana posibles transacciones fraudulentas mediante un modelo híbrido que combine reglas heurísticas y técnicas de Machine Learning.
4. Reducir la tasa de chargebacks y su posible impacto financiero a los negocios de ecommerce.
5. Proporcionar un sistema de revisión manual asistida, donde el comercio pueda evaluar transacciones catalogadas como sospechosas.

II. Trabajos Relacionados

El desarrollo de los sistemas de detección de fraude en el comercio electrónico se ha ido llevando a cabo en una gran variedad de soluciones comerciales. Fraud-AI se relaciona directamente con plataformas antifraude que, al igual que nuestro sistema, se basan en modelos híbridos que combinan aprendizaje automático mediante Machine Learning y reglas heurísticas. Algunas de estos sistemas son:

A. Stripe Radar

Stripe es una plataforma de infraestructura financiera integral que permite a las empresas aceptar pagos en línea de forma segura. Facilita la gestión de las transacciones globales, facturación, cumplimiento fiscal y, lo que nos interesa, prevención de fraude. Stripe Radar funciona mediante las alertas de fraude, las cuales indican a los prestamistas que su identidad podría haber sido comprometida, además de posibles riesgos asociados a una transacción, como el uso indebido de información o comportamientos sospechosos.

Para generar dichas alertas, Stripe Radar hace uso de modelos de Inteligencia Artificial entrenados con los datos de su propia red. Dichos modelos permiten identificar patrones en tiempo real, combinando un puntaje probabilístico con reglas configurables por el comercio. Además, el sistema se va adaptando continuamente a nuevas formas de fraude mediante aprendizaje automático. *Ver [1].*

B. Riskified

Riskified es una empresa tecnológica israelí-estadounidense especializada en prevención de fraude y gestión de riesgos para el comercio electrónico. Su modelo se basa en machine learning avanzado y un análisis de comportamiento del consumidor, ofreciendo decisiones automáticas de aprobación o rechazo.

Una de sus principales características es la automatización en la toma de decisiones, ya que el sistema puede aprobar o rechazar transacciones en tiempo real con base en el nivel de riesgo calculado, lo que permite a los negocios reducir pérdidas por fraude y evitar rechazos innecesarios de clientes verdaderos. *Ver [2]*

C. Kount

Esta empresa se encarga de ofrecer soluciones de detección y prevención de fraude basadas en inteligencia artificial para pagos realizados en línea, comercio electrónico y verificación de identidad. Kount utiliza un aprendizaje automático, inteligencia artificial y análisis de big data para la evaluación del riesgo de transacciones digitales en tiempo real.

Su plataforma combina señales de comportamiento, datos de dispositivos, ubicación geográfica y datos provenientes de redes globales de fraude, con el fin de determinar la probabilidad de riesgo antes de aprobar o rechazar una operación.

El objetivo de Kount es reducir la pérdida de ingresos o mercancía por parte de los comerciantes cuando se presentan transacciones fraudulentas, enfoque que comparte Fraud-AI, al buscar minimizar el impacto financiero derivado del fraude.

D. Dashboards en sistemas de datos

No se puede hablar acerca de trabajos formales relacionados sin tomar en cuenta a los propios dashboards que son utilizados en los sistemas modernos. Dichos sistemas incorporan herramientas de visualización de datos que permite a los usuarios que hacen uso de ellos interpretar grandes volúmenes de información de una forma concisa, clara y eficiente. Los dashboards interactivos facilitan la toma de decisiones al mostrar los datos clave mediante gráficos o métricas comprensibles. Este enfoque ha sido ampliamente utilizado por las empresas que manejan grandes volúmenes de datos para mejorar la gestión y operación de sus sistemas.

No obstante, a diferencia de las soluciones comerciales mencionadas, Fraud-AI propone una arquitectura modular y flexible, orientada a su adaptación en distintos modelos de negocio electrónico. Asimismo, integra un sistema de monitoreo visual interactivo, la combinación de modelos supervisados y no supervisados de Machine Learning, y un conjunto de reglas heurísticas configurables por el usuario, permitiendo mayor control y transparencia en el proceso de detección. y utilizando la metodología ágil SCRUM.

III. Descripción del Desarrollo del Proyecto Modular

El desarrollo de Fraud-AI se llevó a cabo mediante un enfoque modular y utilizando la metodología ágil SCRUM, lo que permitió facilitar la adaptación a cambios durante el desarrollo del proyecto. Esta metodología contribuyó a una mejor planificación y seguimiento de cada componente del sistema.

De esta forma, la separación de responsabilidades fue más sencilla, permitiendo que cada componente del sistema pudiera avanzar de manera independiente. La arquitectura se diseñó bajo un modelo cliente-servidor, integrando un Backend en Python mediante FastAPI, un frontend web interactivo y un motor de detección de fraude basado en técnicas de Machine Learning. En cuanto a FastAPI, este es un framework moderno especializado para el desarrollo de APIs mediante Python, el cual permite construir servicios de alto rendimiento de una manera rápida, utilizando una validación automática de datos. *Ver [3].*

A. Arquitectura General del Sistema

La arquitectura del proyecto Fraud-AI fue diseñada mediante un enfoque modular basado en el modelo de cliente-servidor, con el simple objetivo de garantizar una mejor estructuración de las distintas responsabilidades, así como una facilidad de mantenimiento y escalabilidad futura. Este enfoque permite que cada componente del sistema pueda tener un crecimiento independiente, sin necesidad de alterar o modificar el funcionamiento global del software.

En niveles generales, el sistema se encuentra compuesto de tres capas conceptuales: la vista hacia el cliente (Frontend), la lógica del negocio (Backend) y el procesamiento inteligente (motor de detección). En la implementación actual, algunas de estas capas se encuentran integradas físicamente dentro del mismo entorno para

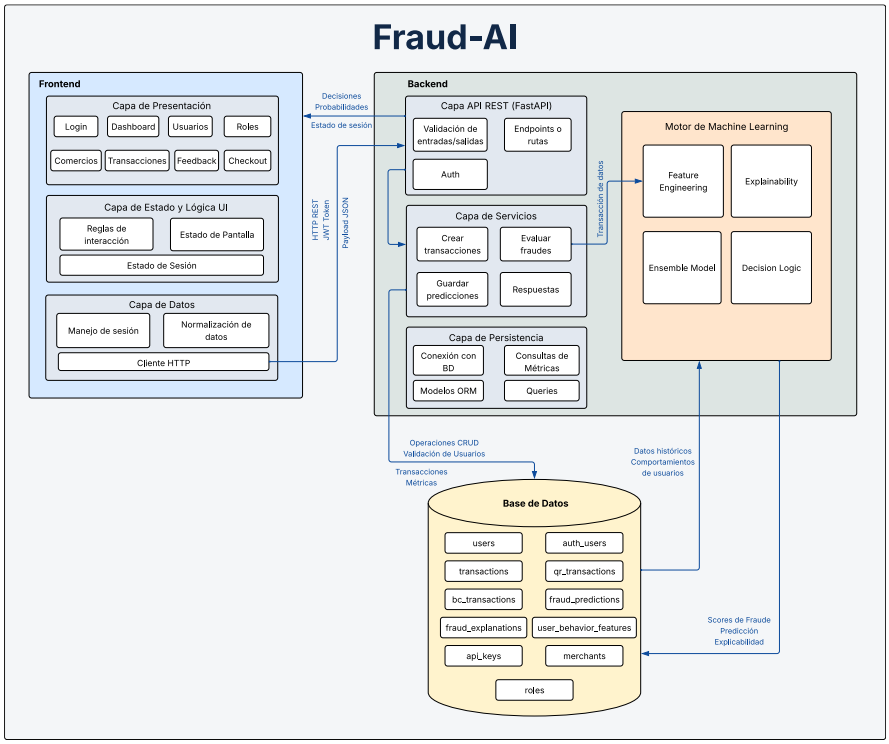

Fig. 1 Arquitectura de software de Fraud-AI

optimizar el desarrollo y el despliegue del sistema, tal como se muestra en la Fig. 1.

El Frontend actúa como la capa de interacción con los usuarios. Es una interfaz web diseñada para permitir la visualización, la consulta y el monitoreo de las transacciones, así como la interpretación de los resultados generados por el sistema de detección de fraude. Dicho componente se comunica directamente con el Backend mediante solicitudes HTTP. Su diseño, desarrollado mediante tecnologías a las cuales se entrará a detalle más adelante, prioriza la claridad de la información, así como la capacidad de actualización dinámica de los datos.

El Backend, desarrollado mediante FastAPI, es el responsable de gestionar toda la lógica del negocio, procesar las solicitudes provenientes desde el Frontend, validar los datos de entrada, mantener una interacción con la base de datos y ejecutar el motor de detección de fraudes. La elección de FastAPI se debe a su alto rendimiento y a su generación automática de documentación, así como la facilidad para estructurar las aplicaciones de una forma modular.

Dentro del Backend se encuentran integrados diversos módulos organizados según sus responsabilidades, tales como:

- Rutas (Endpoints)
- Servicios de Negocio
- Modelos de Datos
- Módulos de Predicción

Dicha organización permite mantener una estructura limpia y modular, donde cada componente cumple con algunas tareas en específico, y a su vez se mantiene una evolución accesible del proyecto, sin generar efectos negativos en otras partes del sistema.

El motor de detección de Fraude basado en técnicas de Machine Learning se encuentra integrado dentro del módulo de Backend, como un módulo especializado. Su función principal es analizar las transacciones recibidas y, mediante el motor de aprendizaje, determinar la probabilidad de que ésta sea fraudulenta.

La comunicación entre los distintos componentes del sistema se realiza mediante una arquitectura basada en *API REST*, lo cual permite una comunicación clara entre Frontend y Backend.

Una API (Application Programming Interface, lo que se traduce a Interfaz de Programación de Aplicaciones) es un conjunto de protocolos y reglas que permiten a distintos componentes de software mantener una comunicación entre ellos. Son fundamentales y necesarios para no mantener en un sólo conjunto todos los módulos de una aplicación, sino que cada componente pueda evolucionar de manera independiente sin que se llegue a afectar el funcionamiento general del sistema.

En el caso de Fraud-AI, se implementó una API de tipo REST (Representational State Transfer, o Transferencia de Estado Representacional), la cual se encuentra basada en el uso de HTTP como el protocolo principal para llevar a cabo la comunicación entre el Frontend y el Backend. Esto permite realizar diversas operaciones, como el envío de nuevas transacciones, consulta de resultados o gestión de usuarios y comercios mediante los endpoints (rutas) definidos en FastAPI, utilizando el formato JSON. *Ver [4].*

Para el almacenamiento de información se hace uso de una Base de Datos Relacional. El Backend actúa como intermediario entre la base de datos y el Frontend, asegurando que todas las interacciones cumplan con las reglas del negocio y las validaciones necesarias.

B. Diseño de la Base de Datos

El diseño de la base de datos de Fraud-AI se desarrolló utilizando un modelo relacional implementado en PostgreSQL, en su versión 16. Éste es un sistema de gestión de bases de datos relacional de código abierto que se caracteriza por su facilidad de uso, su escalabilidad y soporte para transacciones complejas. Es ampliamente utilizado en aplicaciones que requieren integridad de datos y una alta confiabilidad. *Ver [5].*

La estructura fue pensada para que sea capaz de soportar un sistema de detección de fraude en tiempo real, permitiendo almacenar tanto

información transaccional como resultados provenientes de los análisis del motor de detección y del comportamiento del usuario.

El modelo de datos se encuentra normalizado, debido a que cada entidad representa un concepto específico, de tal forma que se evitan redundancias y se facilita la escalabilidad del sistema. A nivel general, la base de datos se organiza en diversas tablas que se podrían agrupar en los siguientes conceptos: gestión de usuarios y comercios, registro de transacciones, predicción de fraude y características de comportamiento.

1) Gestión de Usuarios y Comercios: Para la gestión de usuarios y comercios se cuentan con las tablas *users*, *auth_users*, *merchants* y *roles*. La tabla *users* representa a los usuarios finales que realizan transacciones. Por otro lado, *auth_us*ers corresponde a los usuarios autenticados del sistema que se encuentren utilizando el servicio de Fraud-AI. Esta tabla guarda datos como las credenciales, roles o la asociación directa con algún comercio (*merchants*). Esta separación permite diferenciar entre los usuarios operativos del sistema y los usuarios transaccionales. La tabla *roles* definen los distintos niveles de acceso dentro de cada comercio, implementando un control de permisos personalizado. Y, finalmente, la tabla *merchants* es la encargada de separar los comercios afiliados con Fraud-AI, permitiendo que múltiples negocios utilicen el sistema de manera independiente.

Adicionalmente, se incluye la tabla de api_keys, diseñada específicamente para gestionar el acceso seguro al sistema por parte de los comercios, permitiendo implementar una autenticación por comercio a nivel de API.

2) Registro de Transacciones: Para esta parte se diseñaron tres tablas principales: *transactions*, *qr_transactions* y *bc_transactions*, con el objetivo de diferenciar entre transacciones tradicionales y aquellas realizadas mediante código QR o por métodos de Blockchain. Dichas entidades comparten atributos comunes, como monto, país, hora o día, pero incluyen campos específicos según el tipo de transacción realizada, como información del dispositivo o métricas relacionadas con el uso de QR. Dicha separación permite adaptar mejor los modelos de Machine Learning a las características propias de cada formato de pago.

3) Predicción de Fraude: Para el eje central del sistema se cuenta con la tabla *fraud_predictions*, la cual es la encargada de almacenar los resultados generados por el motor de detección de fraudes. Dicha tabla contiene información específica de cada transacción realizada, como la probabilidad final de fraude, la decisión otorgada por el motor de detección, o los scores individuales de cada algoritmo (regresión logística, random forest y k-means). Mantiene una relación con las tablas de *transactions* y *qr_transactions* mediante el campo de *transaction_id*, así como su campo *channel* que indica el tipo de transacción perteneciente a la predicción. Por otro lado, y para complementar, se hizo uso de la tabla *fraud_explanations*, misma que almacena el aporte individual de cada una de las transacciones en la predicción de fraude. Esto es lo que permite explicar el comportamiento del modelo, facilitando así la toma de decisiones y aportando transparencia total al sistema.

4) Características de Comportamiento: la tabla *user_behavior_features* almacena algunas variables agregadas por cada usuario que realiza alguna transacción, tales como la cantidad de transacciones recientes, intentos fallidos o posibles desviaciones respecto a su comportamiento promedio. Estas características son fundamentales para alimentar los modelos de IA implementados, ya que permite la detección de posibles anomalías basadas en patrones históricos.

C. Desarrollo de Backend y API

El Backend de Fraud-AI representa la parte central del sistema, y fue desarrollado utilizando FastAPI, un framework moderno específico para la construcción de APIs mediante Python, mismo que fue elegido debido a su validación automática de datos, además de su eficiencia y la generación automática de la documentación.

La estructura del Backend fue diseñada siguiendo un enfoque modular, organizando el código en distintas capas que separan las responsabilidades. Estas capas son:

1) Capa de Modelos: Encargada de definir los esquemas, mediante *SQLAlchemy* (biblioteca de kit de herramientas SQL), que representan las entidades del sistema, es decir, las tablas de la base de datos.

2) Capa de Esquemas: Contiene los esquemas *Pydantic*, la cual es una biblioteca de validación de datos en Python. Dichos esquemas aseguran que los datos recibidos en los endpoints sean válidos antes de ser procesados.

3) Capa de Rutas/Endpoints: Es la encargada de definir los puntos de entrada a la API, donde se reciben las solicitudes provenientes del frontend.

4) Capa de Servicios: Implementa la lógica del sistema, incluyendo operaciones como el procesamiento de las transacciones, la evaluación del fraude o la gestión de usuarios.

5) Capa de Consultas: Contiene las consultas SQL que interactúan directamente con la base de datos, y devuelve los resultados a la capa de servicios.

Esta arquitectura permite realizar modificaciones al sistema sin afectar directamente a las demás capas. Por otro lado, dentro el mismo Backend se encuentra integrado el motor de detección de fraude, lo cual permite realizar predicciones en tiempo real con menor latencia, ya que de esta manera no es necesario realizar llamadas externas para evaluar cada transacción.

El Backend presenta múltiples rutas o endpoints organizados según su funcionalidad, entre los cuales se pueden encontrar:

1) Gestión de Autenticación de Usuarios: Registro, inicio de sesión y control de acceso mediante roles de usuario.

2) Gestión de Comercios: Administración de los negocios que utilizan Fraud-AI.

3) Recepción de Transacciones: Endpoints especializados para los distintos tipos de transacciones que acepta el sistema.

4) Consulta de Predicciones: Permite obtener los resultados históricos y en tiempo real.

5) Retroalimentación de Fraude: Registro de etiquetas reales para la retroalimentación del sistema.

Por otro lado, fue implementado un sistema de autenticación basado en API Keys, permitiendo a los comercios consumir los servicios de manera segura y personal. Estas claves son almacenadas en la base de datos en formato hash de manera segura, y son únicas para cada negocio asociado con Fraud-AI, de tal forma que estos puedan tener los servicios de detección de fraude para sus respectivos clientes.

En cuanto a los términos de comunicación con el Frontend, la API sigue los principios de REST (Representational State Transfer), utilizando métodos HTTP estándar, mismos que son los métodos de GET, POST, PUT y DELETE, así como los datos estructurados en formato JSON. Esto asegura la compatibilidad con múltiples clientes y facilita la integración con aplicaciones externas, y además está preparado para ser desplegado mediante servidores ASGI, tal como lo es Uvicorn, lo cual permite manejar múltiples solicitudes concurrentes de una manera eficiente.

D. Implementación del Motor de Detección

El motor de detección de fraude fue implementado como un módulo especializado basado en técnicas de *Machine Learning*, la cual es una disciplina de la inteligencia artificial centrada en el desarrollo de modelos capaces de aprender a partir de los datos, mediante la identificación de patrones que permiten realizar predicciones, lo cual en nuestra problemática de la detección de fraude es de gran utilidad debido a la necesidad de analizar grandes volúmenes de datos para encontrar patrones ocultos en los mismos. *Ver [6].*

Es el engrane de en medio que permite el funcionamiento de todo el conjunto. Su principal objetivo es el de analizar las transacciones ocurridas en tiempo real a los comercios asociados con Fraud-AI, y determinar la probabilidad de que estas mismas sean fraudulentas. Se encuentra integrado dentro del Backend, lo que permite analizar las transacciones de forma eficiente.

En cuanto al diseño, la arquitectura del motor de detección fue construida bajo un enfoque modular, nuevamente permitiendo la escalabilidad futura. Además, tiene un enfoque híbrido, en el cual contribuyen los resultados individuales de múltiples algoritmos, los cuales son:

1) Regresión Logística: Es un modelo supervisado que genera la probabilidad de fraude según los datos de entrada, interpretando relaciones lineales entre las variables de las transacciones. La regresión logística es un modelo utilizado en problemas de clasificación binaria, ya que permite obtener la probabilidad de que cierto evento ocurra a partir de un conjunto de variables de entrada. *Ver [7].*

2) Random Forest: Este algoritmo permite capturar relaciones no lineales y patrones complejos. Su estructura permite mejorar la robustez y reducir posibles sobreajustes. Random Forest se basa en la combinación de distintos árboles de decisión, lo cual ayuda a mejorar la precisión de las predicciones al reducir posibles sobreajustes. Es útil en problemas en donde los datos presentan comportamientos no lineales. *Ver [7].*

3) K-Means: Es un algoritmo no supervisado que permite detectar anomalías al agrupar las transacciones en *clústeres*, asignando puntajes de riesgo según qué tan alejadas se encuentren del comportamiento promedio. Este algoritmo es útil para encontrar las transacciones con un comportamiento inusual que se alejan de la normalidad, considerándose posibles anomalías. *Ver [8].*

Cada uno de estos modelos mencionados generan puntajes individuales, según una combinación de factores, para posteriormente generar una evaluación más potente del riesgo de fraude, reduciendo riesgos individuales de cada modelo por separado, y mejorando la precisión de la decisión final.

El flujo de la operación del motor de detección es el siguiente:

1. El sistema recibe los datos desde el Frontend mediante un endpoint REST, donde también se valida la identidad del comercio a través de una API Key y se verifican los datos de entrada.

2. Después, los datos son procesados y enriquecidos con información adicional referente al usuario, como los intentos fallidos, actividad durante las últimas 24 horas, promedio de montos y variables de contexto.

3. El sistema verifica la existencia del usuario que se encuentra realizando la transacción y, en caso de no existir, se genera dicho nuevo usuario.

4. Luego, ya con todo preparado, los datos son evaluados por cada uno de los modelos del motor de Machine Learning, generando diferentes las probabilidades y sus respectivos puntajes de riesgo.

5. Posteriormente, se combinan las salidas de los distintos modelos utilizados mediante un enfoque de tipo ensemble, con el fin de generar una probabilidad de fraude final más completa.

6. Ya teniendo estos resultados, se aplica una lógica de decisión que clasifica la transacción en las diferentes posibles salidas: *revisión*, *permitir* o *bloquear*, considerando tanto los resultados del modelo como las reglas de negocio.

7. Después, tanto los resultados obtenidos son almacenados en la base de datos, para tener una posibilidad de mantener un posterior análisis.

8. Finalmente, y si es que el nivel de riesgo lo amerita, se generan explicaciones del modelo, además de actualizarse la información de comportamiento del usuario para futuras evaluaciones, terminando con el proceso de operación del motor de detección de fraude.

Un aspecto que tomar en cuenta del diseño es el añadido de diversas métricas dentro del modelo de datos, como probabilidades individuales por cada algoritmo o los puntajes adicionales. Esto permite mejorar tanto la toma de decisiones como el análisis de comportamiento de cada modelo de manera independiente.

En cuanto a las explicaciones de las decisiones, el sistema cuenta con mecanismos para poder determinar qué variables influyeron en la decisión del modelo, indicando valores tanto del peso de cada variable como la dirección de su impacto, si fue positiva o negativa. Esto permite tener una transparencia total y decisiones justificadas. Además, la implementación del motor de decisión de fraude se encuentra acoplada a los mecanismos de seguridad del Backend, como la autenticación del comercio, la persistencia en la base de datos, la generación de notificaciones para los casos de *revisión* y de *bloqueo*, o la generación de las métricas y diagramas para el monitoreo continuo.

Ahora bien, el flujo completo de aprendizaje permite tanto el entrenamiento de los modelos de detección de fraude como una mejora continua a partir de una retroalimentación operativa. Para el proceso de entrenamiento, se tomó un dataset de aproximadamente 300,000 transacciones, en el cual existían aproximadamente un 2% de resultados fraudulentos. Este problema de desbalance es común en la detección de fraude, ya que la cantidad de transacciones legítimas supera con creces a las fraudulentas. Esta situación puede afectar el desempeño de los modelos al momento de su entrenamiento, ya que se tiende a favorecer la clase mayoritaria, por lo que fue necesario la implementación de estrategias para obtener resultados confiables. *Ver [9].*

Para ello, se implementaron los modelos de Regresión Logística para la captura de relaciones lineales y una buena interpretabilidad, Random Forest para la detección de interacciones no lineales entre las variables, y K-Means para la identificación de anomalías o comportamientos no típicos. Las salidas de estos modelos fueron combinadas mediante un metamodelo basado en la técnica de *stacking*, el cual consiste en integrar diversos modelos base y utilizar sus predicciones como entrada de un modelo adicional que se encarga de acoplarlos para optimizarlos de una mejor manera. Esto permite aprovechar las fortalezas de cada modelo individual, generando un puntaje final más robusto. *Ver [7].*

Por otra parte, se incorporan técnicas de explicabilidad mediante la Explicación Aditiva de Shapley, o por sus siglas, *SHAP*. Dichas técnicas permiten interpretar las predicciones realizadas por los modelos. Se basan en los valores que contribuyen a obtener los resultados, midiendo el impacto de cada variable en la decisión final, lo cual es necesario para comprender por qué una transacción fue clasificada como fraudulenta o legítima. Esta interpretabilidad es necesaria, ya que permite validar el comportamiento del modelo, así como hacerle saber a los comercios acerca de las razones por las cuales el sistema detectó posibles transacciones fraudulentas en su negocio. *Ver [10].*

Finalmente, para la retroalimentación del sistema se incluye un mecanismo mediante el cual los comercios pueden indicar si una predicción fue correcta, permitiendo detectar patrones de error del modelo, identificar ciertos casos en los que el sistema fallo, y generar datos reales para futuros reentrenamientos. Todo esto permite al sistema evolucionar con el tiempo.

E. Desarrollo del Frontend y Monitoreo Visual

El análisis de los datos es fundamental en la toma de decisiones de los comercios. En este caso, analizar las transacciones permite identificar patrones, tendencias y comportamientos anormales que pueden indicar actividades sospechosas.

La representación visual de estos datos mediante gráficos y dashboards facilita su interpretación, permitiendo que los usuarios representantes de los comercios comprendan rápidamente la información y actúen en la medida de lo posible. De esta manera, no solamente se detectan los riesgos, sino que también se apoya en la toma de decisiones. *Ver [11].*

Esta parte visual de Fraud-AI fue desarrollada como una interfaz web enfocada en la visualización y análisis de las transacciones en tiempo real. Su objetivo es el de presentar de manera clara y sencilla los resultados generados por el sistema, de tal forma que se les facilite la toma de decisiones a los usuarios que hagan uso de este.

El Frontend se comunica directamente con el Backend mediante APIs REST y los servicios de FastAPI. Esta forma de comunicación permite enviar las transacciones, consultar predicciones de fraude y visualizar el historial de las transacciones de manera dinámica, utilizando formatos como JSON.

En cuanto a las tecnologías utilizadas para el Frontend, se empleó Next.js como framework principal, ya que permite construir una interfaz tanto moderna como escalable, además de una buena integración con el Backend. *Ver [12].* Para el desarrollo de los componentes visuales se utilizó React junto con TypeScript, lo que permitió mejorar la organización del código, el tipado de los datos y la mantenibilidad del proyecto. Además, se implementó Tailwind CSS para el diseño de la interfaz, permitiendo desarrollar un estilo responsivo y consistente en diferentes tamaños de pantalla.

Por otro lado, se utilizaron bibliotecas especializadas para mejorar la experiencia del monitoreo y la visualización de los datos. Una de estas librerías es *Recharts*, orientada a la creación de gráficas interactivas, la cual permitió representar las tendencias de fraude, el comportamiento de las transacciones y los indicadores del sistema mediante gráficos dinámicos. Este tipo de herramientas son necesarias para aplicaciones de análisis de datos, ya que la visualización de la información permite transformar grandes volúmenes en representaciones comprensibles por los usuarios. *Ver [13].*

Otra librería es *Radix UI*, biblioteca enfocada en la creación de componentes accesibles y reutilizables, tales como selectores, diálogos, pestañas, menús, alertas y notificaciones, lo cual permitió construir interfaces consistentes y alineadas con buenas prácticas y mejoras en cuanto a la interpretabilidad.

Asimismo, *Lucide React* añade los íconos usados en la interfaz, y *Sooner* se utiliza para mostrar las notificaciones emergentes del sistema, permitiendo informar al usuario acerca de alertas importantes, como lo son los resultados de nuevas transacciones junto con sus respectivas predicciones.

Por otro lado, *react-hook-form* y *Zod* son usados para el manejo y la validación de los formularios, lo cual permitió implementar validaciones eficientes en los datos ingresados por los usuarios, reduciendo errores, agregando nuevas capas de seguridad y mejorando la experiencia al hacer uso del sistema. *Ver [14].*

La estructura del frontend fue diseñada de manera modular, separando los componentes, las páginas y la lógica de consumo de servicios.

Algunas de las funcionalidades principales del sistema son:

1) Dashboard: Muestra el resumen general del sistema, incluyendo transacciones recientes, métricas clave y la distribución de posibles decisiones (*permitir*, *revisión*, *bloquear*).

2) Visualización de Transacciones: Cada transacción incluye información como el monto, el canal de procesamiento, la probabilidad de fraude, la decisión generada por el sistema y el nivel de riesgo. Esta información se presenta de manera visual para facilitar su interpretación y permitir una revisión rápida.

3) Revisión y seguimiento: El sistema cuenta con una sección específica para revisar transacciones marcadas como bloqueadas o en revisión. Desde esta vista, que se puede apreciar en la Fig. 2, el usuario puede filtrar por estado y por método de pago, consultar el detalle de cada caso y dar seguimiento a las operaciones en las que se necesite atención.

4) Alertas y notificaciones: Cuando se detecta una transacción con alta probabilidad de fraude, la interfaz es capaz de mostrar avisos visuales para que el usuario tome una decisión informada de forma eficiente. Esto mejora la capacidad de respuesta por parte de los comercios ante posibles riesgos. En la Fig. 3 se puede observar la vista del dashboard en conjunto con el panel lateral del menú de notificaciones.

5) Gestión de usuarios y comercios: También se incorporan módulos para la administración de usuarios, comercios, roles y perfil de usuario, lo cual permite controlar de forma adecuada los diferentes aspectos operativos del sistema.

En conjunto, el Frontend de Fraud-AI fue diseñado para poder ofrecer al usuario final una experiencia sencilla, funcional y orientada a la toma de decisiones, permitiendo consultar información relevante de manera rápida y visual.

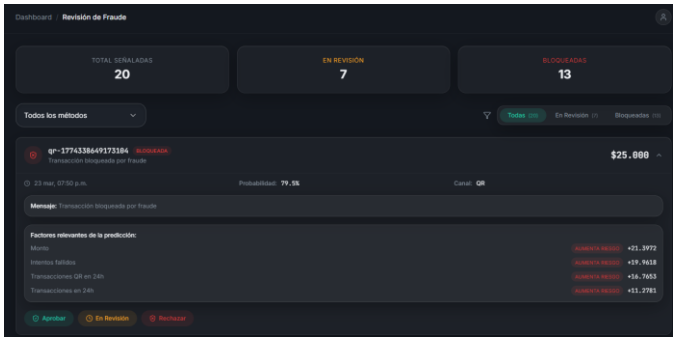

Fig. 2 Vista principal del apartado de revisión y seguimiento, en donde se puede observar los factores relevantes de la predicción que influyeron en la toma de decisión del sistema.

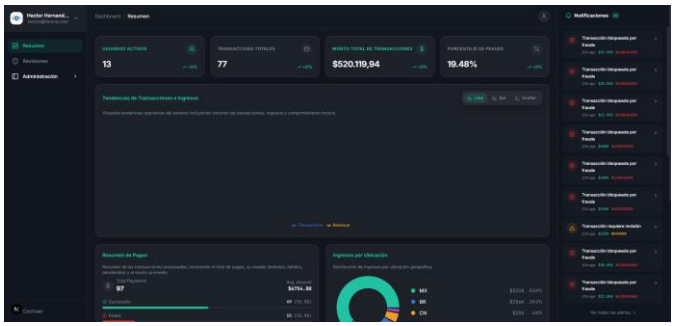

Fig. 3 Vista principal del dashboard en conjunto con el menú de notificaciones.

F. Despliegue y Gestión en Servidor

El proyecto fue implementado bajo un esquema de despliegue en la nube que permite separar de manera clara los distintos componentes del sistema, específicamente el frontend, el Backend y la base de datos. Esta organización facilitó una administración más eficiente de la infraestructura, así como una mejor escalabilidad y disponibilidad del sistema. Para ello, el Backend desarrollado con FastAPI fue desplegado en Railway, al igual que la base de datos PostgreSQL, mientras que el frontend construido con Next.js fue alojado en Vercel, garantizando así el acceso remoto a la plataforma desde cualquier entorno con conexión a internet. Estas plataformas son servicios orientado a aplicaciones modernas. Mientras que Vercel está enfocada principalmente al despliegue de aplicaciones Frontend, especialmente aquellas desarrolladas mediante Next.js, Railway permite desplegar servicios de Backend y bases de datos de manera eficiente mediante contenedores y variables de entorno. Estas plataformas son consideradas PaaS (Plataformas como Servicios). *Ver [15], [16].*

En el proceso de gestión en servidor se configuraron variables de entorno para resguardar credenciales y parámetros sensibles, evitando su exposición directa en el código fuente. Asimismo, se implementaron mecanismos de seguridad y control de acceso basados en JWT, API Keys y cookies seguras para el manejo de la sesión, además de configuraciones de CORS para permitir la comunicación controlada entre el frontend y el Backend.

El despliegue en servidor también permitió validar la integración completa entre los módulos del sistema en un entorno real, verificando el funcionamiento de los procesos de autenticación, consulta de datos, administración de comercios, usuarios y roles, así como la visualización de métricas y alertas.

G. Aplicación de la Metodología SCRUM

La metodología SCRUM fue el marco organizativo que permitió el desarrollo evolutivo del proyecto, facilitando de esta forma la adaptación a los cambios realizados y la validación de los avances del sistema. *Ver [17].*

1) Roles: En cuanto al rol de *Product Owner*, no se centró en una persona como tal, sino en la definición de los requerimientos a cubrir y las tareas a priorizar. Así mismo, un miembro del equipo tomó la responsabilidad del *Scrum Máster*, con el fin de facilitar todo el proceso de desarrollo mientras se aseguraba de que la metodología fuera cumplida, dejando al resto del equipo con el rol de *Development Team,* encargado de implementar las funcionalidades del sistema.

2) Sprints: El trabajo fue separado en sprints, mismos que fueron ciclos cortos de desarrollo enfocados en la entrega de funcionalidades específicas. Por ejemplo, en los primeros sprints se priorizaron componentes base como la estructura del propio Backend, el modelo de datos y la autenticación. Luego, en sprints posteriores se integró el motor de fraude y el procesamiento de las transacciones, y finalmente se implementó el Frontend con el sistema de monitoreo.

3) Ceremonias ágiles: Se realizaron actividades tales como la planeación de los sprints en las que se definieron las tareas a desarrollar. Además, se llevaron a cabo reuniones de seguimiento en las que se permitió evaluar el avance general del proyecto, identificando problemáticas presentadas y puntos de mejora. Así mismo, se realizaron revisiones de sprint, en las cuales se verificaba que las funcionalidades implementadas en el sistema cumplieran con lo acordado en sus respectivas planeaciones.

De manera general, el sistema de Fraud-AI integra una gran variedad de componentes que, trabajando en conjunto, ofrecen un servicio completo de detección de posible fraude. La combinación de una arquitectura modular, los modelos de Machine Learning y las herramientas de visualización en el Frontend permiten a los usuarios tanto identificar riesgos como gestionarlos de la manera en que mejor se les acomode a ellos como negocio.

Módulo I Gestión de la tecnología de información

Fraud-AI se relaciona con este módulo al implementar un sistema de información completo orientado a la detección de fraude, usando principios de ingeniería de software que garantizan la calidad, seguridad y mantenibilidad del sistema. Para su desarrollo se hizo uso de la metodología SCRUM, lo cual permitió estructurar el proyecto en capas, abarcando desde la construcción de las bases del sistema hasta el desarrollo e implementación del motor de inteligencia y del sistema de monitoreo visual.

El sistema fue diseñado mediante capas, separando el Frontend del Backend y la base de datos, lo cual facilitó la organización del código y su escalabilidad. Además, se implementaron mecanismos de validación de datos, control de acceso mediante JWT y API Keys, así como explicabilidad de las predicciones generadas. En cuanto a los estándares y las herramientas, se emplearon protocolos como HTTP/HTPS, arquitectura REST, formato JSON y tecnologías como FastAPI, SQLAlchemy, PostgreSQL y Next.js. Asimismo, se eligieron lenguajes como Python para el Backend y Machine Learning, TypeScript/JavaScript para el frontend y SQL para la gestión de datos, garantizando un equilibrio entre el rendimiento y su mantenibilidad.

Módulo II Sistemas robustos, paralelos y distribuidos

El proyecto cumple con este módulo mediante el diseño de una arquitectura basada en un modelo cliente-servidor, el cual permite la distribución de responsabilidades entre los diferentes componentes del sistema. Además, el sistema integra múltiples algoritmos de detección de fraude, lo que demuestra dominio en la implementación de soluciones complejas. Por otro lado, se encuentra alojado en un servidor en la nube, cumpliendo con el requisito de la distribución. La arquitectura del sistema se encuentra construida de tal forma que cada componente pueda escalar de forma independiente, además de poder soportar mayores volúmenes de datos y de usuarios sin afectar el rendimiento general del sistema.

Módulo III Justificación de Cómputo Flexible (softcomputing)

Fraud-AI se relaciona con el módulo III al cubrir tres ramas de inteligencia artificial directamente: la primera rama es la de Machine Learning, ya que se utilizaron los algoritmos de Regresión Logística y Random Forest para poder entrenar el sistema utilizando un dataset de 300,000 registros; la segunda rama corresponde a Arboles de Decisión, representados a través del modelo Random Forest, el cual permite capturar relaciones más complejas entre sus variables; y la tercera rama es la Minería de Datos, al analizar los patrones de comportamiento de los usuarios y detectar posibles anomalías en las transacciones.

El modelo implementado combina sus enfoques mediante un esquema híbrido, en donde cada algoritmo es capaz de aportar perspectivas diferentes que se integran en un modelo final para generar un score de fraude más completo.

IV. Resultados Obtenidos Del Proyecto

El desarrollo de Fraud-AI como proyecto modular permitió la implementación de un sistema funcional orientado a la detección de fraude en transacciones bancarias, cumpliendo en su totalidad con los objetivos planteados inicialmente.

1. Se desarrolló una plataforma web, orientada como un SaaS, que es capaz de permitir a los comercios tener un mayor control sobre las transacciones en su negocio, al integrar un sistema capaz de analizar las operaciones y asignar posibles niveles de riesgo. Esto facilitó la identificación de posibles transacciones fraudulentas antes de que llegaran a tener un impacto económico mayor, tanto en los negocios como en los clientes.
2. Se implementó un sistema de monitoreo visual mediante un dashboard interactivo, el cual permitió representar gráficamente los resultados históricos de las transacciones realizadas a cada comercio, así como ciertas métricas de fraude y patrones relevantes, lo cual ayudó a la comprensión por parte de los usuarios y facilitó la toma de decisiones basada en dichos datos.
3. En cuanto a la detección de fraude, se integró un modelo que combinó técnicas de Machine Learning junto con reglas heurísticas, que fue capaz de identificar transacciones sospechosas. Como resultado, el sistema generó predicciones con un nivel de precisión adecuado, utilizando un dataset apto para su entrenamiento.
4. Se estableció un sistema de decisión capaz de clasificar las transacciones en diferentes niveles, lo cual ayudó a la reducción potencial de posibles chargebacks a los comercios al detectar operaciones riesgosas antes de su confirmación.
5. Se implementó un sistema de revisión manual asistida, en el cual los usuarios tienen la oportunidad de analizar transacciones sospechosas y proporcionar retroalimentación sobre las decisiones finales, lo cual permitió tanto validar los resultados del modelo generado como mantener una mejora continua del mismo.

Los resultados obtenidos demostraron que la solución propuesta fue efectiva para el análisis, el monitoreo y la prevención de transacciones fraudulentas, cumpliendo con los objetivos definidos y validando la viabilidad del sistema en un ámbito real.

V. Conclusiones y Trabajo a Futuro

Después de haber concluido la realización del proyecto modular, se comprobó que el sistema implementado logró detectar transacciones fraudulentas, mostrar las métricas relevantes y permitir al usuario tomar una decisión final basada en los resultados generados por el modelo de IA desarrollado. De esta forma, los objetivos propuestos fueron cumplidos en su totalidad, ya que se validó que la solución propuesta resuelve las problemáticas planteadas, y el prototipo realizado cumple con su objetivo.

Se pudo observar la efectividad de un modelo híbrido que combina tanto reglas heurísticas como técnicas de Machine Learning. La integración de múltiples algoritmos, como Regresión Logística, Random Forest y K-Means, mejoró la precisión en comparación con el caso en el que se hubiera utilizado un solo modelo.

Por otro lado, la separación de la arquitectura del proyecto en las capas de Frontend, Backend y modelos de IA fue fundamental para el desarrollo del sistema, debido a que de esta manera se facilitó en enorme medida el crecimiento de este, reduciendo la dependencia entre los distintos módulos y mejorando la organización general del proyecto.

En conjunto, el desarrollo del sistema permitió integrar diversos conceptos de ingeniería de software, bases de datos e inteligencia artificial en un solo sistema funcional orientado a una problemática real.

En cuanto a trabajo a futuro, se identificaron diversas áreas de mejora para el crecimiento y fortalecimiento del sistema. En primer lugar, se plantea la integración de datos reales provenientes de negocios existentes, lo cual permitiría mejorar la precisión de los modelos de detección. También se considera implementar modelos más avanzados, como lo son las redes neuronales, con el único fin de mejorar la detección de fraude, reduciendo el margen de error al momento de clasificar las transacciones.

Además, se contempla ampliar la capacidad del sistema para manejar un mayor volumen de transacciones y comercios, manteniendo un rendimiento adecuado a medida que vaya creciendo el sistema.

Por último, se considera la opción de seguir ampliando las funcionalidades del sistema, ya sea incorporando nuevos tipos de análisis o de métricas que permitan a los comercios que hagan uso de Fraud-AI tener una mejor comprensión del comportamiento de sus clientes y tomar decisiones más informadas.

Reconocimientos

Se agradece y reconoce de manera especial a Alan Uriel Pérez Cisneros, quien participó en la creación inicial de la idea de este proyecto y formaría parte del equipo de trabajo. Su aportación fue significativa en las primeras etapas del desarrollo.

Referencias

[1] Stripe, *"Cómo funciona una alerta de fraude y cuándo usarla,"* 2026. [Online]. Available: https://stripe.com/mx/resources/more/fraud-alerts-how-they-work-and-when-to-use-them

[2] Riskified, *"Riskified,"* [Online]. Available: https://www.riskified.com/es

[3] FastAPI, *"FastAPI Documentation,"* 2024. [Online]. Available: https://fastapi.tiangolo.com/

[4] [16] IBM, *"¿Qué es una API (Interfaz de Programación de Aplicaciones)?"*, https://www.ibm.com/mx-es/topics/api

[5] PostgreSQL Global Development Group, *"PostgreSQL Documentation,"* 2024. [Online]. Available: https://www.postgresql.org/docs/

[6] S. Theodoridis, Machine Learning: A Bayesian and Optimization Perspective. Elsevier, 2015.

[7] G. James, D. Witten, T. Hastie, and R. Tibshirani, An Introduction to Statistical Learning. Springer, 2013.

[8] J. Han, M. Kamber, and J. Pei, Data Mining: Concepts and Techniques. Morgan Kaufmann, 2011.

[9] H. He and E. A. Garcia, *"Learning from imbalanced data,"* IEEE Transactions on Knowledge and Data Engineering, 2009.

[10] S. Lundberg and S. Lee, *"A Unified Approach to Interpreting Model Predictions,"* Advances in Neural Information Processing Systems (NeurIPS), 2017.

[11] More Than Digital, *"Explicación de Data-Driven Decision-Making (DDDM),"* [Online]. Available: https://insights.mtd.info/es/explicacion-de-data-driven-decision-making-dddm-introduccion-significado-ejemplos-y-el-proceso-de-decisiones-basadas-en-datos/

[12] Vercel, "Next.js Documentation," 2024. [Online]. Available: https://nextjs.org/docs

[13] Tableau, *"Mejores prácticas de visualización de datos y análisis,"* [Online]. Available: https://www.tableau.com/learn/articles/data-visualization-best-practices

[14] React Hook Form, *"React Hook Form Documentation,"* 2024. [Online]. Available: https://react-hook-form.com

[15] Vercel, *"Vercel Documentation,"* 2024. [Online]. Available: https://vercel.com/docs

[16] Railway, *"Railway Documentation,"* 2024. [Online]. Available: https://docs.railway.app

[17] K. Schwaber and J. Sutherland, *"The Scrum Guide,"* 2020. [Online]. Available: https://scrumguides.org